CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 6

Rendering External JSON Data using Fetch API, Search and Filter

JavaScript (ES6+) · Fetch API · JSON · HTML5 · CSS3 · PHP 8 (page and access control)

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 6 report (`Practical-6/PRAC_6_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report described the standalone page `Practical-6/directory.html` (five ES modules, served by Python's http.server). This version describes the Campus Data Explorer of the portal – `studenthub/data_explorer.php` with `studenthub/assets/js/data.js` – which reads the same anonymised Practical 6 datasets (copied into `studenthub/data/`) but is simpler than the standalone page; every difference is listed in Section 10. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To display students, events and FAQs from external JSON files inside the StudentHub portal using the JavaScript Fetch API, with dynamic rendering, search, filtering, sorting and pagination, and with clear loading and error states.

- Load three JSON datasets (students, events, FAQs – Practical 6's sample data, 15 records each) with `fetch()` and render them as tables.
- Implement search, filter, sort and pagination with JavaScript array methods (`filter`, `some`, `includes`, `sort`, `slice`, `forEach`) and a `Set` for the filter options.
- Show a loading message while a file is being fetched and an error message with the reason when loading fails.
- Show each file's own "SAMPLE DATA" description, so the user always sees that the records are invented.
- Escape every value from the JSON files before it is inserted into the page.
- Make the page a normal part of the portal: portal layout and sidebar link, available to signed-in users only.

The advanced extension of the standalone version (a localStorage cache of the last response) and its ES-module structure were not carried over; see Section 10.

2. Problem Definition

Official problem definition: "Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination."

Practical 6 was first built as the standalone page `Practical-6/directory.html`. In the integrated portal the audit in `TEST_REPORT.md` (section 12) found that "`assets/js/data.js` and `data/*.json` existed but **no page used them**". The integration therefore added the Campus Data Explorer page `data_explorer.php`; at the same time `data.js` was changed to escape every value, and the artificial 800 ms delay it contained was removed. After the report review the portal's earlier JSON files were replaced by Practical 6's own anonymised sample datasets (`DEMO-001` … `DEMO-015`), and `data.js` was extended to read their format and to show each file's description above the table (`TEST_REPORT.md` section 13).

The practical list asks four key questions; the answers for the integrated implementation are:

Key question	Answer in the integrated portal
How is JSON fetched, parsed, and rendered?	`fetchData()` calls `fetch()` with the template literal `data/${type}.json`, checks `response.ok`, parses the body with `response.json()`, keeps the `records` list in `currentData` and shows the file's `description` with `showNote()`. `renderTable()` builds the table head from the keys of the first record and one row per record, with

	every value escaped by `esc()`.
Which array methods are used for search, filter, sort, and pagination?	`filter()` with `Object.values(item).some()` and `includes()` for the search, `filter()` with equality on `branch` (students) or `category` (events and FAQs) for the filter, `sort()` with `localeCompare()` for the A–Z / Z–A order, `slice()` for the 5 rows of the current page, and `forEach()` with a `Set` to build the filter options.
How are loading and error states handled?	`showLoading()` shows "⏳ Fetching Data from JSON..." in `#status-container` (an `aria-live="polite"` region); `showError()` shows "Error loading data:" followed by the reason, with `role="alert"`; when nothing matches, "No records found." is shown.
How is modularity maintained in JavaScript files?	All behaviour is in one file, `data.js`, inside a `DOMContentLoaded` handler, split into functions with one task each (`fetchData`, `populateFilterOptions`, `applyFilters`, `renderTable`, `renderPagination`, `showNote`, `showLoading`, `showError`). The PHP page contains only the empty containers. Unlike the standalone version, it is not split into ES modules.

3. Tools and Technologies

Tool / technology	Use in this practical
HTML5 + PHP 8.2.12 (XAMPP)	`studenthub/data_explorer.php`: page structure, access check, sidebar
CSS3	`studenthub/assets/css/global.css`: tabs, toolbar, status messages and table of the Data Explorer
JavaScript (ES6+)	`studenthub/assets/js/data.js`: Fetch API, async/await, array methods, template literals
JSON	`studenthub/data/students.json`, `events.json`, `faqs.json` – Practical 6's sample datasets, 15 records each
Python 3 (standard library)	`tests/test_forms.py`: HTTP tests against the PHP built-in server
Playwright + Microsoft Edge (headless)	`tests/browser_forms.py`: browser checks and the screenshots in Section 8
Git / GitHub	Version control; the integration is commit `938e74f` of the StudentHub repository, the later switch to the Practical 6 datasets is described in `TEST_REPORT.md` section 13
Operating system	Windows 11

4. Place in the Integrated StudentHub Portal

The Data Explorer is a normal portal page in the *Campus* group of the sidebar ("Data Explorer", `sidebar.php` line 23), next to Notices, the Event Finder, the Directory and Help & Contact. It uses the portal's layout, theme and session.

File / route	Role
`studenthub/data_explorer.php` (route `data_explorer.php`)	The page: header, "Sample data" note, three dataset buttons, search / filter / sort controls and the containers for status, table and pagination. Requires the permission `portal.view`.
`studenthub/assets/js/data.js`	The Practical 6 script (loaded with `defer`).
`studenthub/data/students.json`, `events.json`, `faqs.json`	The external JSON datasets – byte-for-byte copies of `Practical-6/data/*.json`.
`studenthub/assets/css/global.css`	Styles under the comment "Campus Data Explorer (Practical 6)".
`studenthub/sidebar.php`	The sidebar link.
`tests/test_forms.py`,	Tests of the integration (Section 9).

tests/browser_forms.py	

Access is controlled by the portal's integrated login (Practicals 9 and 10): `require_permission("portal.view")` admits the roles admin, editor and student and sends a visitor who is not signed in to the login page.

studenthub/data_explorer.php (lines 2–7) – purpose of the page and access check

```
2  // StudentHub – Campus Data Explorer (Practical 6 integrated): the sample JSON datasets in data/
   (students, events, FAQs) are
3  // loaded with the Fetch API by assets/js/data.js and shown with search, filter, sorting and
   pagination. The live portal
4  // records are in the Directory (students) and the Event Finder (events); these files demonstrate
   client-side rendering.
5  require_once "db.php";
6  require_once __DIR__ . "/includes/auth.php";
7  $current_user = require_permission("portal.view");
```

The note on the page states that the records are sample data and points to the pages with the live database records: the Directory (students) and the Event Finder (`events_live.php`, Practical 13), which also uses `fetch()` but reads the database through the portal's JSON API `api/events.php`. The Data Explorer complements it by demonstrating client-side rendering of static JSON files.

5. Implementation

5.1 The page and its containers

The three buttons carry the dataset name in `data-type`. The page itself contains no data: the table, the dataset description, the status messages and the page buttons are produced by `data.js`. Without JavaScript the `noscript` message "The Data Explorer needs JavaScript." is shown.

studenthub/data_explorer.php (lines 37–41) – the three dataset buttons

```
37              <div class="explorer-tabs" role="group" aria-label="Dataset">
38                  <button type="button" class="tab-btn active" data-type="students">Students</button>
39                  <button type="button" class="tab-btn" data-type="events">Events</button>
40                  <button type="button" class="tab-btn" data-type="faqs">FAQs</button>
41              </div>
```

Between the buttons and the containers (lines 42–47) is the toolbar with three labelled controls: a search box (`type="search"`, id `searchInput`, with autocomplete switched off), the filter select `#filterSelect` (first option "All Categories/Branches") and the sort select `#sortSelect` with the options Default order, Name A–Z and Name Z–A. Then follow the containers:

studenthub/data_explorer.php (lines 48–51) – status, table and pagination containers and the noscript message

```
48              <div id="status-container" aria-live="polite"></div>
49              <div id="data-table-container"></div>
50              <div id="pagination-container" class="pagination-bar"></div>
51              <noscript><p class="alert-error">The Data Explorer needs JavaScript.</p></noscript>
```

5.2 JSON datasets

The three files are identical copies of the datasets of the standalone `Practical-6/data/` folder. Each is a JSON object that labels itself as sample data: `dataset` (its name), `sampleData: true`, `description` (a sentence beginning "SAMPLE DATA - …") and `records`, a list of 15 objects. The student file, for example, describes itself as "SAMPLE DATA - fictional students created for ITUE203 Practical 6. All names, IDs (DEMO-001 to DEMO-015) and CPI values are invented and do not belong to real people."

File	Records	Fields	Filter	Sorted by
`students.json`	15	id (DEMO-001 … DEMO-015),	branch (CE, CSE, IT)	name

		name, branch, semester, cpi		
events.json	15	id, title, date (yyyy-mm-dd), venue, category	category (Cultural, Hackathon, Seminar, Social, Sports, TechFest, Workshop)	title
faqs.json	15	id, category, question, answer	category (Academics, Attendance, Campus, Exams, Fees, Hostel, Portal)	question

5.3 Fetching and parsing the JSON (Fetch API)

`fetchData()` is an `async` function: it shows the loading state, awaits the response, treats any status outside 200–299 as an error (`response.ok`) and parses the JSON. It then takes the `records` list (a plain array would be accepted as it is), shows the file's `description` with `showNote()`, and builds the filter options from the data with a `Set`, so each branch or category appears once. Any exception – a failed request, a bad status or invalid JSON – is caught and shown as an error message. The same function is called when the page loads (students) and whenever a dataset button is clicked; a click also clears the search and resets the sort order.

studenthub/assets/js/data.js (lines 41–76) – fetchData() and populateFilterOptions()

```
41      async function fetchData(type) {
42          showLoading();
43          try {
44              const response = await fetch(`data/${type}.json`);
45              if (!response.ok) throw new Error('Failed to fetch data');
46
47              const data = await response.json();
48              // Practical 6 format: {dataset, sampleData, description, records: [...]} (a plain array
    is accepted too)
49              currentData = Array.isArray(data) ? data : (data.records || []);
50              showNote(Array.isArray(data) ? '' : (data.description || ''));
51
52              populateFilterOptions(type);
53              applyFilters();
54          } catch (error) {
55              showError(`Error loading data: ${error.message}`);
56          }
57      }
58
59      // Populate Filter Dropdown dynamically based on data type
60      function populateFilterOptions(type) {
61          filterSelect.innerHTML = '<option value="all">All Categories/Branches</option>';
62          let options = new Set();
63
64          currentData.forEach(item => {
65              if (type === 'students') options.add(item.branch);
66              if (type === 'events' || type === 'faqs') options.add(item.category);
67          });
68
69          options.forEach(opt => {
70              if (opt) {
71                  filterSelect.innerHTML += `<option value="${esc(opt)}">${esc(opt)}</option>`;
72              }
73          });
74      }
75
76      // Filter, Search, and Sort Logic
```

5.4 Search, filter and sorting

`applyFilters()` runs on every key press in the search box and on every change of the filter or sort select. It never changes `currentData`; it builds `filteredData` from it and always returns to page 1.

- **Search:** case-insensitive (`toLowerCase()`), matched against every value of a record – including the ID, numbers such as the semester, and for FAQs the answer text.
- **Filter:** equality on `branch` (students) or `category` (events and FAQs).

- **Sort:** “Name A–Z” / “Name Z–A” compare the `name`, `title` or `question` field with `localeCompare()`; “Default order” keeps the order of the file.

studenthub/assets/js/data.js (lines 77–112) – search, filter and sort

```
 77      function applyFilters() {
 78          const searchTerm = searchInput.value.toLowerCase();
 79          const filterVal = filterSelect.value;
 80          const sortVal = sortSelect.value;
 81
 82          filteredData = currentData.filter(item => {
 83              // Search logic (check values of object)
 84              const matchesSearch = Object.values(item).some(val =>
 85                  String(val).toLowerCase().includes(searchTerm)
 86              );
 87
 88              // Filter logic
 89              let matchesFilter = true;
 90              if (filterVal !== 'all') {
 91                  if (currentType === 'students') matchesFilter = item.branch === filterVal;
 92                  if (currentType === 'events' || currentType === 'faqs') matchesFilter = item.category
    === filterVal;
 93              }
 94
 95              return matchesSearch && matchesFilter;
 96          });
 97
 98          // Sorting logic
 99          if (sortVal !== 'default') {
100              filteredData.sort((a, b) => {
101                  let valA = a.name || a.title || a.question || a.q;
102                  let valB = b.name || b.title || b.question || b.q;
103                  valA = String(valA).toLowerCase();
104                  valB = String(valB).toLowerCase();
105
106                  if (sortVal === 'name_asc') return valA.localeCompare(valB);
107                  if (sortVal === 'name_desc') return valB.localeCompare(valA);
108              });
109          }
110
111          currentPage = 1; // Reset to first page
112          renderTable();
```

5.5 Dynamic rendering

`renderTable()` computes the number of pages, takes the 5 records of the current page with `slice()`, and builds the table: one `<th scope="col">` per key of the first record (upper-cased) and one row per record. Each value passes through `esc()` before it is placed in the HTML string; a CPI of 9 or more is additionally wrapped in `<mark>`.

studenthub/assets/js/data.js (lines 116–155) – renderTable()

```
116      function renderTable() {
117          statusContainer.innerHTML = '';
118
119          if (filteredData.length === 0) {
120              tableContainer.innerHTML = '<p style="padding:20px; text-align:center;">No records
    found.</p>';
121              paginationContainer.innerHTML = '';
122              return;
123          }
124
125          // Pagination calculations
126          const totalPages = Math.ceil(filteredData.length / itemsPerPage);
127          const startIndex = (currentPage - 1) * itemsPerPage;
128          const paginatedData = filteredData.slice(startIndex, startIndex + itemsPerPage);
129
130          // Generate Table HTML
131          let html = '<div class="table-wrap"><table class="data-table"><thead><tr>';
132
133          // Dynamic Headers
134          const keys = Object.keys(paginatedData[0]);
135          keys.forEach(key => {
```

```
136             html += `<th scope="col">${esc(key.toUpperCase())}</th>`;
137         });
138         html += '</tr></thead><tbody>';
139 
140         // Dynamic Rows
141         paginatedData.forEach(item => {
142             html += '<tr>';
143             keys.forEach(key => {
144                 let val = esc(item[key]);
145                 // Styling specific columns
146                 if (key === 'cpi' && item[key] >= 9) val = `<mark>${val}</mark>`;
147                 html += `<td>${val}</td>`;
148             });
149             html += '</tr>';
150         });
151 
152         html += '</tbody></table></div>';
153         tableContainer.innerHTML = html;
154 
155         renderPagination(totalPages);
```

5.6 Pagination

`renderPagination()` creates Previous, one button per page and Next; Previous is disabled on the first page and Next on the last; the current page button gets the class `active`. When all results fit on one page, no page buttons are shown.

studenthub/assets/js/data.js (lines 159–189) – renderPagination()

```
159     function renderPagination(totalPages) {
160         if (totalPages <= 1) {
161             paginationContainer.innerHTML = '';
162             return;
163         }
164 
165         let html = `<button id="prevBtn" ${currentPage === 1 ? 'disabled' : ''}>Previous</button>`;
166 
167         for (let i = 1; i <= totalPages; i++) {
168             html += `<button class="page-btn ${currentPage === i ? 'active' : ''}" data-
    page="${i}">${i}</button>`;
169         }
170 
171         html += `<button id="nextBtn" ${currentPage === totalPages ? 'disabled' : ''}>Next</button>`;
172         paginationContainer.innerHTML = html;
173 
174         // Pagination Events
175         document.getElementById('prevBtn')?.addEventListener('click', () => {
176             if (currentPage > 1) { currentPage--; renderTable(); }
177         });
178         document.getElementById('nextBtn')?.addEventListener('click', () => {
179             if (currentPage < totalPages) { currentPage++; renderTable(); }
180         });
181         document.querySelectorAll('.page-btn').forEach(btn => {
182             btn.addEventListener('click', (e) => {
183                 currentPage = parseInt(e.target.getAttribute('data-page'));
184                 renderTable();
185             });
186         });
187     }
188 
189     // The dataset's own description (e.g. "SAMPLE DATA - fictional students ..."), inserted as text
```

5.7 Dataset description, loading and error states

`showNote()` creates a `<p id="dataset-note">` above the table once and writes the file's description into it with `textContent`, so the text can never be interpreted as HTML; the paragraph is hidden when a file has no description. The loading message is placed in `#status-container`, which is an `aria-live="polite"` region, and the table and page buttons are cleared while a file is loading. The error message is escaped, because it contains the text of the caught exception.

studenthub/assets/js/data.js (lines 189–212) – showNote(), showLoading() and showError()

```
189      // The dataset's own description (e.g. "SAMPLE DATA - fictional students ..."), inserted as text
190      function showNote(text) {
191          let note = document.getElementById('dataset-note');
192          if (!note) {
193              note = document.createElement('p');
194              note.id = 'dataset-note';
195              note.className = 'section-note';
196              tableContainer.before(note);
197          }
198          note.textContent = text;
199          note.hidden = !text;
200      }
201
202      // UI States
203      function showLoading() {
204          tableContainer.innerHTML = '';
205          paginationContainer.innerHTML = '';
206          statusContainer.innerHTML = '<div class="loading">⏳ Fetching Data from JSON...</div>';
207      }
208
209      function showError(msg) {
210          tableContainer.innerHTML = '';
211          paginationContainer.innerHTML = '';
212          statusContainer.innerHTML = `<div class="error-message" role="alert">⚠️ ${esc(msg)}</div>`;
```

6. Database and Relationships

Practical 6 uses no database table. The Data Explorer reads three static JSON files with `fetch()`; nothing is written and nothing is stored in the browser. The only server-side rule is the access check of `data_explorer.php`: `require_permission("portal.view")` uses the signed-in session (the role comes from `users.role`).

The live student and event records of the portal remain in the `students` and `events` tables of `studenthub_db` and are shown on the Directory and the Event Finder pages. The JSON files contain only invented sample records and are not connected to those tables, which is why the page labels them as sample data and links to the live pages.

7. Validation and Security

- **Escaping:** `esc()` replaces &, <, >, " and ' with HTML entities. It is applied to every table value, every column heading, every filter option and the error text, so text in a JSON file can never run as HTML; the dataset description is inserted with `textContent`. The escaping was added during the integration (`TEST_REPORT.md` section 12: "`data.js` now escapes every value").
- **Response checks:** a status outside 200–299 and invalid JSON both end in the error message instead of a broken page.
- **Fixed URLs:** the dataset name in the request comes from the `data-type` attribute of the three buttons in the page, not from free user input.
- **Access:** the page requires sign-in. The JSON files themselves are static files that the server delivers without a session – the HTTP test fetched them without signing in – so they must never contain private data; they contain only the invented sample records described in Section 5.2.

studenthub/assets/js/data.js (lines 4–10) – the esc() function and the state of the explorer

```
4   document.addEventListener('DOMContentLoaded', () => {
5       const esc = (value) => String(value).replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;',
      '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
6       let currentData = [];
7       let filteredData = [];
8       let currentType = 'students';
9       let currentPage = 1;
10      const itemsPerPage = 5;
```

8. Output Screenshots

All screenshots are genuine full-page captures of `studenthub/data_explorer.php` in Microsoft Edge (headless, driven by Playwright, viewport 1280 × 900 px), made by `tests/browser_forms.py` on 31 October 2026 against the PHP built-in server and a disposable database, after the switch to the Practical 6 datasets. The signed-in user “Riya Shah” (26IT041) is a fictional test account created by the test. For the loading screenshot the test holds the request for `data/students.json` in its own network layer until the screenshot has been taken and then releases it, so the response is the real file; this capture replaces an earlier one that had been saved only after the data had loaded (`TEST_REPORT.md` section 13).

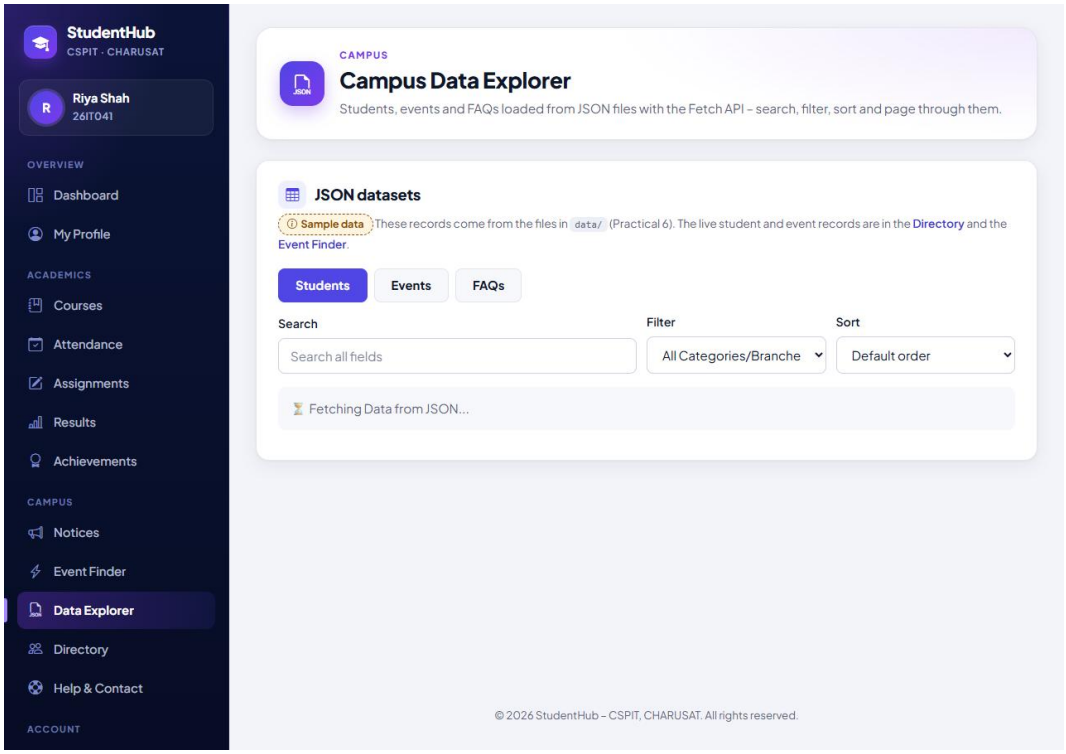


Figure 1: Loading state: while the request for data/students.json is still pending, the status area shows “ Fetching Data from JSON...” below the controls; no table, description or page buttons are shown yet. (04-data-explorer-loading.png)

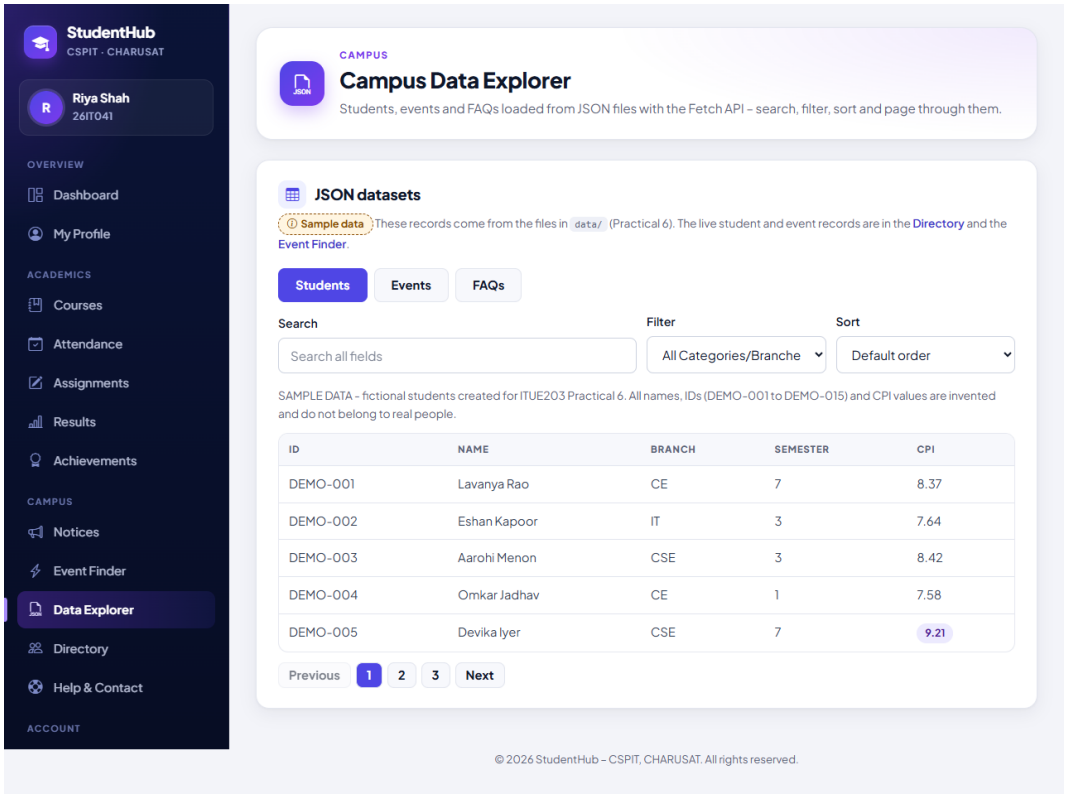


Figure 2: Students dataset loaded: the file's own description ("SAMPLE DATA - fictional students created for ITUE203 Practical 6. All names, IDs (DEMO-001 to DEMO-015) and CPI values are invented …") above the table; page 1 of 3 with DEMO-001 to DEMO-005, columns taken from the JSON keys (ID, NAME, BRANCH, SEMESTER, CPI), and the CPI 9.21 highlighted with <mark>. (05-data-explorer-students.png)

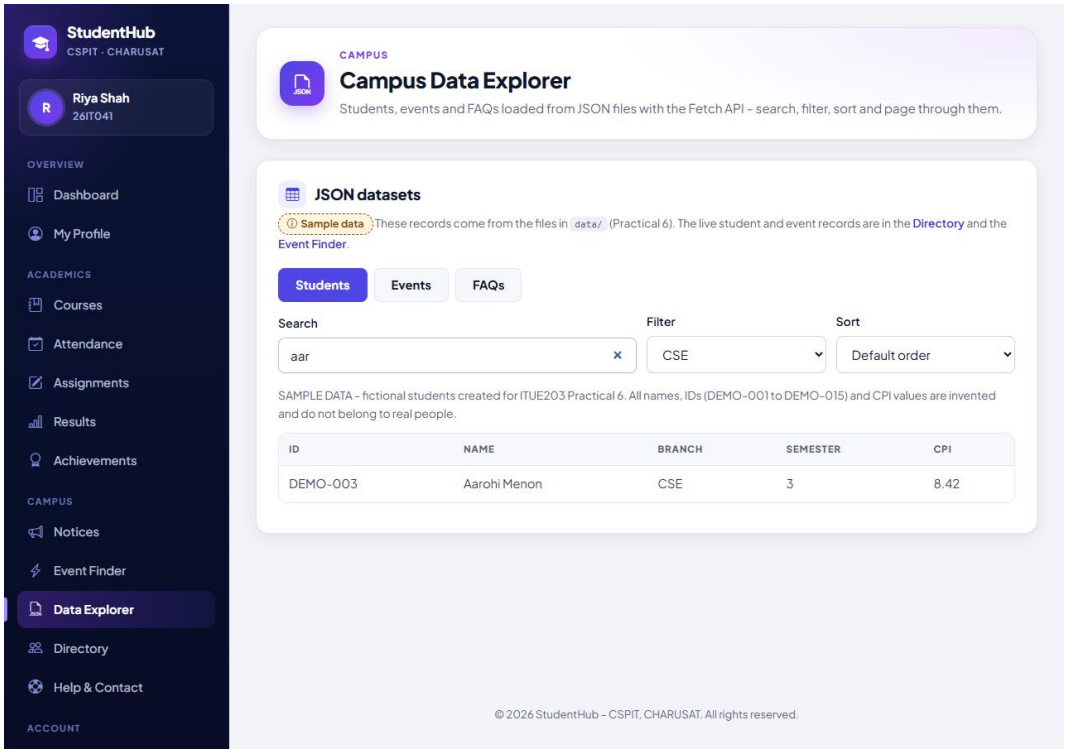


Figure 3: Search "aar" (a term the test derives from the data) combined with the branch filter CSE: exactly one record remains, DEMO-003 Aarohi Menon (CSE, semester 3, CPI 8.42); no page buttons are shown because the result fits on one page. (06-data-explorer-search-filter.png)

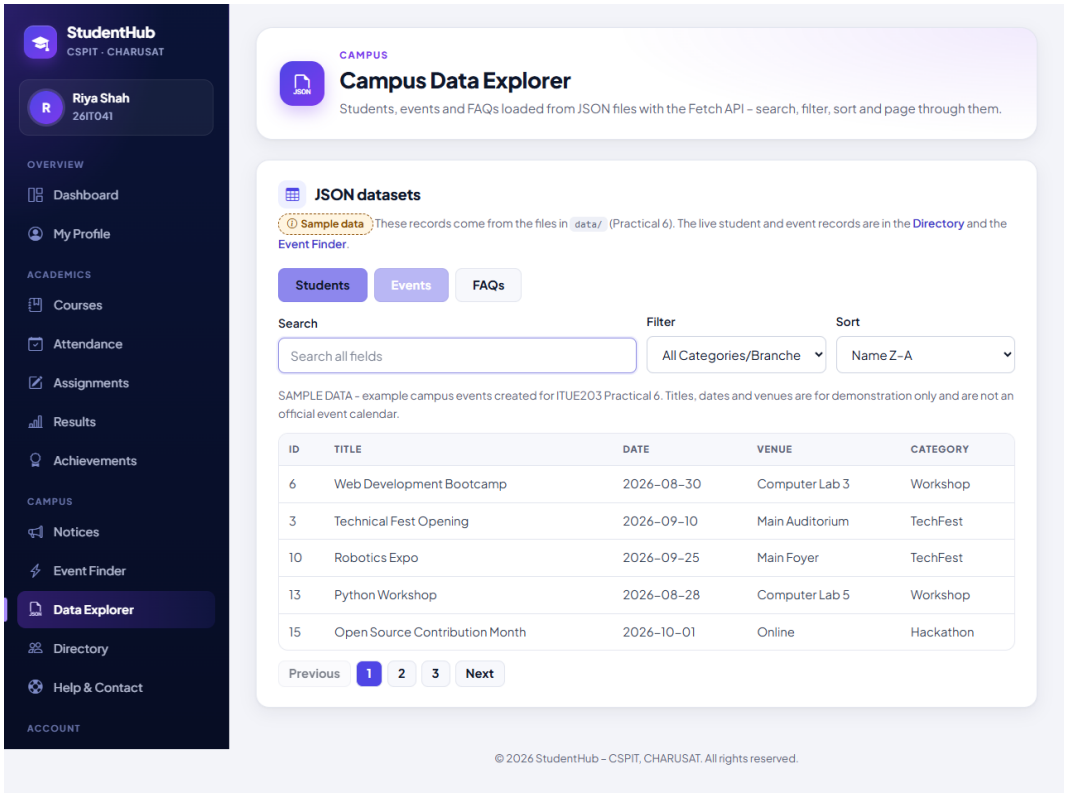


Figure 4: Events dataset (ID, TITLE, DATE, VENUE, CATEGORY) with its own SAMPLE DATA description, sorted "Name Z–A" by title: Web Development Bootcamp, Technical Fest Opening, Robotics Expo, Python Workshop, Open Source Contribution Month on page 1 of 3. (07-data-explorer-events-sorted.png)

9. Testing and Results

Environment of the recorded runs (31 October 2026): Windows 11, PHP 8.2.12 (XAMPP) built-in server on 127.0.0.1, MariaDB 10.4.32 for the disposable test database, Microsoft Edge in headless mode through Playwright. The complete run of all suites after these changes (`tests/run_all_output.txt`) passed 841 checks with 0 failures.

Suite	Scope	Result	Output file
`tests/test_forms.py`	HTTP tests of Practicals 5, 6 and 7 in the portal; 6 checks concern Practical 6	**41 / 41 passed**	`tests/test_forms_output.txt`
`tests/browser_forms.py`	Microsoft Edge checks and screenshots of Practicals 5–7; 5 checks concern Practical 6, two more cover the whole run (no JavaScript errors, no PHP warnings)	**15 / 15 passed**	`tests/browser_forms_output.txt`

Practical 6 checks, copied from the saved outputs:

- data_explorer.php (signed in) has the tabs, search, filter, sort, status, table and pagination containers and loads data.js – PASS
- not signed in: redirected to the login page – PASS
- the sidebar links to the Data Explorer – PASS
- data/students.json, events.json and faqs.json are Practical 6's sample datasets (sampleData, description, 15 records each) – PASS
- the student dataset uses invented DEMO-001 … DEMO-015 ids (no real enrollment numbers) – PASS

- data.js fetches the JSON, handles errors, escapes every value and paginates (5 per page) – PASS
- Edge: Data Explorer: loading state while students.json is fetched (no table yet) – PASS
- Edge: the dataset's own 'SAMPLE DATA' description is shown above the table – PASS
- Edge: students rendered from JSON: 5 rows per page with pagination – PASS
- Edge: search 'aar' + branch filter CSE -> exactly the 1 matching CSE student(s) – PASS
- Edge: events tab sorted Z–A – PASS
- Edge (whole run): no JavaScript errors; no PHP warnings or errors in the server log – PASS

Note on the method: the sixth HTTP check searches the source of `data.js` for `fetch`, the use of `data.records`, `response.ok`, the `catch` block, the escaping of each value and the page size, and confirms that the old 800 ms delay is gone; the behaviour is checked in Edge.

9.1 Not tested

- The error state was not produced in a browser; it is covered only by the source check above, so there is no screenshot of an error message.
- The FAQs dataset (and its category filter) was checked over HTTP but not opened in Edge.
- Keyboard-only use, screen readers, browsers other than Edge and narrow (mobile) widths.

10. Differences from the Original Standalone Practical and Limitations

Aspect	Standalone Practical-6 (original report)	Integrated portal
Page and server	`directory.html`, Python http.server on port 8606, no login	`data_explorer.php` in the portal; sign-in required
Code structure	five ES modules (config, api, filters, render, main)	one script, `data.js`, with separate functions
Data	objects labelled as sample data with a `records` list; fictional DEMO-001… IDs	the same three files (byte-identical copies); the description is shown above the table
Search	listed fields only, spaces trimmed, matches highlighted with <mark>	all values of a record, not trimmed, no highlighting
Filter	branch, event category, FAQ category	the same: branch, event category, FAQ category
Sorting	several fields in both directions, `aria-sort`	one text field (name / title / question), A–Z or Z–A
Pagination	5, 10 or 15 rows; `aria-current`	fixed 5 rows per page
Loading / errors	spinner, `aria-busy`, 8-second time-out, Retry button, error simulation panel	text message; error with the reason; no Retry button and no time-out
Cache (advanced)	last response kept in localStorage	none
Fast switching	request number + AbortController guard	no guard against a slow, older response
Tests	29 static, 22 contrast and 93 Edge checks	6 HTTP and 5 Edge checks within the 41 / 41 and 15 / 15 suites

10.1 Limitations

- **JavaScript required:** without JavaScript the page shows only the notice “The Data Explorer needs JavaScript.”
- **Sample data:** the JSON files are static sample files, not the portal's live records, and they are not kept in step with the database.
- **Public files:** the JSON files can be downloaded without signing in, although the page itself requires it; they contain only invented data.
- **No retry or cache:** after a failed request the user must click the dataset button again or reload the page.

- **Accessibility:** the dataset buttons are plain buttons in a `role="group"` (not the ARIA tabs pattern of the standalone version), and the sorted column is not marked with `aria-sort`.

11. Conclusion

The StudentHub portal now contains the Campus Data Explorer, `studenthub/data_explorer.php`, which loads Practical 6's anonymised students, events and FAQs datasets with the Fetch API and renders them as tables with search, filter, A–Z / Z–A sorting and pagination of 5 rows, all built with array methods. Each file's own "SAMPLE DATA" description is shown above the table, loading and error states are shown in a live region, every value from the files is HTML-escaped, and the page is reachable from the sidebar for signed-in users only.

The integrated version is deliberately simpler than the standalone one: the localStorage cache, the ES-module structure, the time-out and Retry button and the extra accessibility features remain in `Practical-6/`. The integration passed the HTTP suite (41 / 41) and the Edge suite (15 / 15), and the screenshots show the loading state, the loaded data with its description, a combined search and filter, and a sorted events list.

12. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 6: "Rendering External JSON Data using Fetch API, Search and Filter", CSPIT, CHARUSAT.
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – folders `studenthub/` (portal), `Practical-6/` (original standalone version and its datasets), `tests/`, `evidence/practical-05-07-integration/`; `TEST_REPORT.md` sections 12 and 13.
- MDN – Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN – Response.ok: https://developer.mozilla.org/en-US/docs/Web/API/Response/ok
- MDN – Array.prototype.filter(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- MDN – String.prototype.localeCompare(): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/localeCompare
- MDN – Node.textContent: https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent
- MDN – ARIA live regions: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/ARIA_Live_Regions